

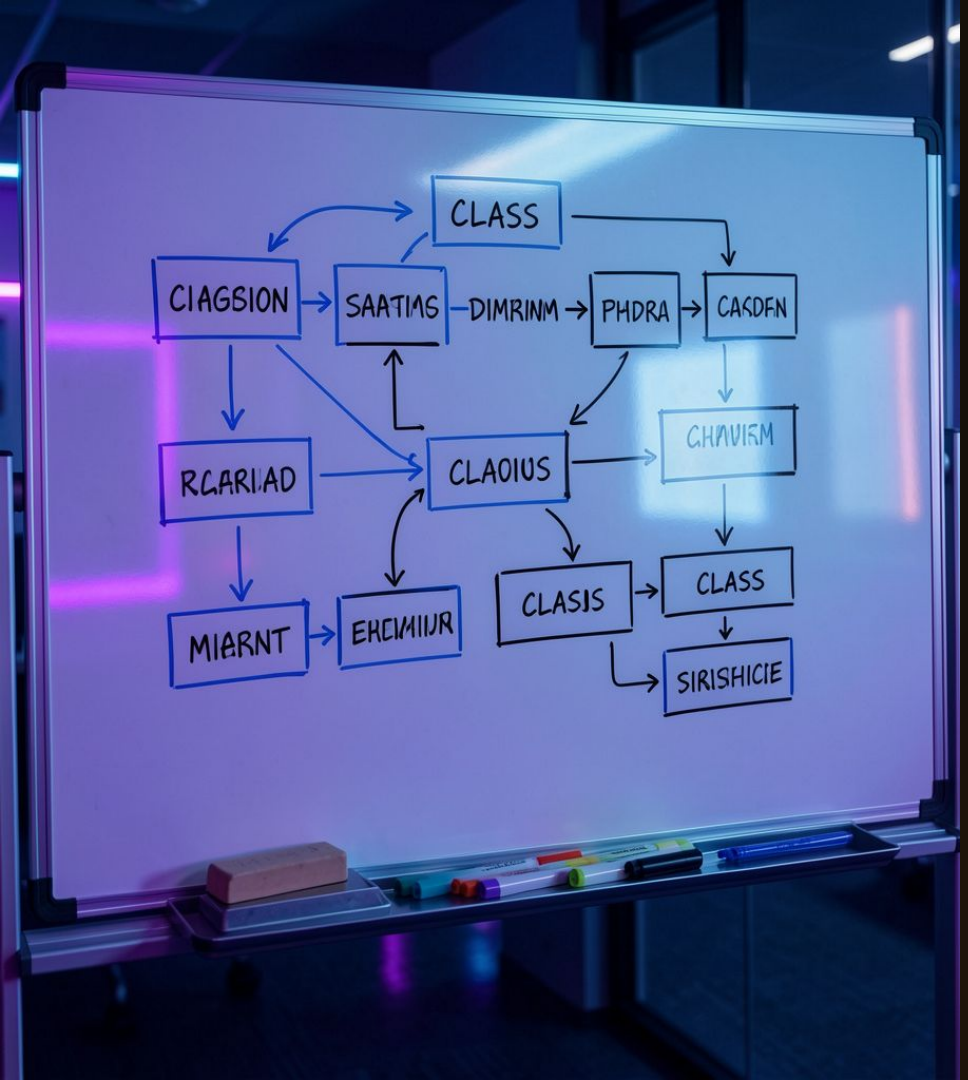
POO Básica

Aula 13: conceitos essenciais e exemplos práticos

Introdução

Aula rápida para entender os pilares da *Programação Orientada a Objetos* e como aplicá-los no dia a dia.

Neste módulo veremos *Classes*, *Objetos*, *Encapsulamento* e *Construtores* com exemplos simples.



01

Classes

Definição e propósito

Uma *classe* é o molde que define atributos e comportamentos de um tipo de objeto.

Serve para organizar código, promover reutilização e modelar entidades do mundo real. **Abstração** é a ideia central.

Atributos e métodos

Atributos representam o estado; *métodos* representam ações.

Use nomes claros e coesos. **Separação de responsabilidades** ajuda na manutenção e testabilidade.



Tipos de acesso

Modificadores controlam visibilidade: *public*, *private*, *protected*.

Encapsular com **getters/setters** protege o estado e permite validações ao alterar dados.

02

Objetos





Criar um objeto é instanciar uma classe: new Classe().

Cada instância tem seu próprio estado; compartilham o comportamento definido pela classe.

Instanciação

Estado e comportamento

Estado = valores dos atributos; comportamento = execução dos métodos.

Projetar bem evita estados inconsistentes e facilita evolução do software.

Referências e memória

Referências apontam instâncias na memória; variáveis guardam endereços, não cópias.

Entenda *heap* e *stack* para gerenciar ciclo de vida e evitar vazamentos. **Referências nulas** e compartilhamento de estado são fontes comuns de bugs.



03

Encapsulamento

Getters e setters

Use *getters* e *setters* para acessar atributos privados com controle.

Permitem validação, logging ou transformação ao ler/escrever. **Evite expor** o estado interno diretamente.

Modificadores de acesso

Modificadores controlam visibilidade: *public*, *private*, *protected*.

Escolha o mais restrito possível para reduzir acoplamento e proteger invariantes da classe.



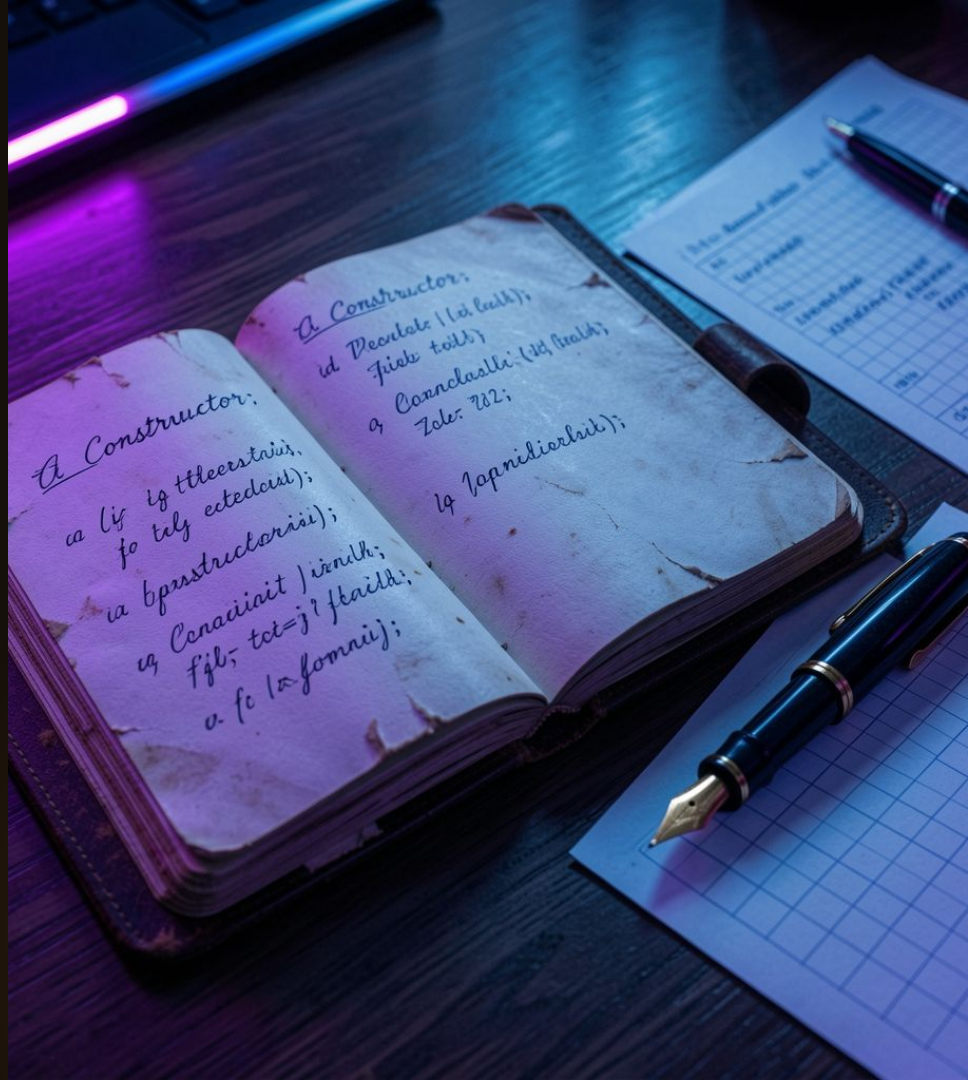
Benefícios e exemplos

Encapsulamento melhora manutenção, segurança e testes.

Exemplo: validar idade no *setter* para garantir estado consistente e evitar checagens repetidas.

04

Construtores



Construtor padrão

Construtor padrão inicializa instâncias sem parâmetros.

Útil para frameworks e criação simples; verifique sempre valores padrão para evitar estados inválidos.



Permite múltiplas formas de criar objetos com diferentes parâmetros.

Oferece flexibilidade – delegue para um construtor principal para evitar duplicação de código.

Sobrecarga de construtores

Inicialização segura

Garanta que todos os atributos críticos sejam inicializados no construtor.

Use validações, exceções e *factory methods* quando a criação exigir lógica complexa. **Evite objetos parcialmente construídos**.

Conclusões

POO organiza código em modelos compreensíveis: *classes*, *objetos* e encapsulamento são centrais.

Construtores e controle de acesso mantêm objetos consistentes; pratique boas inicializações e use referências com atenção.